МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

Тема: Поиск с возвратом

Студентка гр. 4343 _____ Геращенкова М.Д.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 11 до N−1N−1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера NN . Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×77×7 может быть построена из 9 обрезков.

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Описание алгоритма

Для решения задачи используется итеративный поиск с возвратом.

Поле размера N × N представляется не в виде двумерной матрицы, а в виде массива высот heights, где heights[j] показывает, до какой высоты уже заполнен j-й столбец. Такое представление позволяет быстро определять первую незаполненную позицию и максимально возможный квадрат, который можно туда поставить.

На каждом шаге алгоритм:

находит минимальную высоту среди всех столбцов;

выбирает самый левый столбец с этой высотой;

определяет максимальный размер квадрата, который можно поставить в эту позицию;

перебирает размеры квадрата от большего к меньшему;

после установки квадрата переходит к следующему состоянию;

если дальнейшее продолжение ветви не может улучшить найденный результат, выполняется откат.

Так как использование рекурсии запрещено, вместо рекурсивных вызовов применяется явный стек состояний.

Каждое состояние хранит информацию о текущей позиции, допустимых размерах квадрата и данных, необходимых для отката.

Для ускорения работы используются отсечения:

если текущее число квадратов уже не меньше лучшего найденного ответа, ветвь отбрасывается;

вычисляется нижняя оценка числа квадратов, необходимых для заполнения оставшейся площади;

используется запоминание уже посещённых состояний.

Двумерное хранение поля требует постоянной проверки большого числа клеток. Использование массива высот позволяет компактно представлять текущее состояние разбиения и быстро находить первую незаполненную область. Это уменьшает накладные расходы на проверку допустимости очередного шага. Итеративная реализация со стеком полностью соответствует требованию использовать поиск с возвратом без рекурсии.

Выполнение работы.

Используемые структуры данных

heights — массив длины N, описывающий заполненность столбцов;

current — список квадратов в текущей ветви поиска;

best — список квадратов в лучшем найденном решении;

stack — стек состояний для итеративного бэктрекинга;

memo — словарь для хранения уже посещённых состояний.

Алгоритм работает по следующей схеме:

1. Считывается значение N.
2. Если N чётно, квадрат сразу разбивается на 4 квадрата размера N/2.
3. Иначе создаётся начальное состояние:

- массив высот заполняется нулями;
- формируется начальная верхняя граница жадным методом;
- в стек заносится корневое состояние.

4. Пока стек не пуст:

- берётся верхнее состояние;
- если допустимые размеры в нём закончились, выполняется откат;
- иначе выбирается очередной размер квадрата;
- квадрат временно добавляется в текущее решение;
- если поле заполнено полностью, решение сравнивается с лучшим найденным;
- если ветвь бесперспективна, выполняется откат;
- иначе в стек добавляется следующее состояние.

5. После завершения перебора выводится лучшее найденное разбиение.

Исследование.

Алгоритм основан на поиске с возвратом, поэтому его временная сложность в худшем случае является экспоненциальной. Это связано с тем, что число возможных разбиений квадрата быстро растёт при увеличении N, и в общем случае приходится рассматривать большое количество состояний.

По времени сложность можно оценить как экспоненциальную:

O(c^n^2),c>1

По памяти:

без учёта таблицы посещённых состояний — O(n^2), так как хранятся массив высот, стек состояний и текущее решение;

с учётом memo память в худшем случае также может быть экспоненциальной.

Рисунок 1. Зависимость времени выполнения от N.

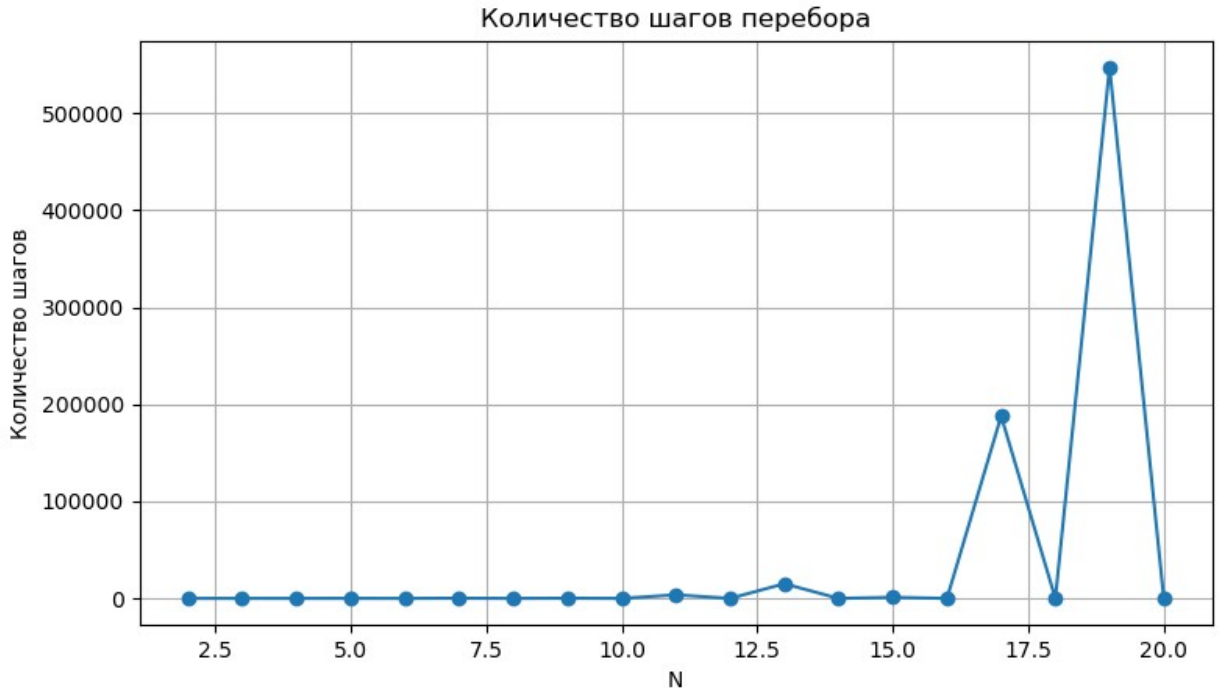

Рисунок 2. Зависимость количества шагов перебора от N.

Рисунок 3. Зависимость глубины стека от N.

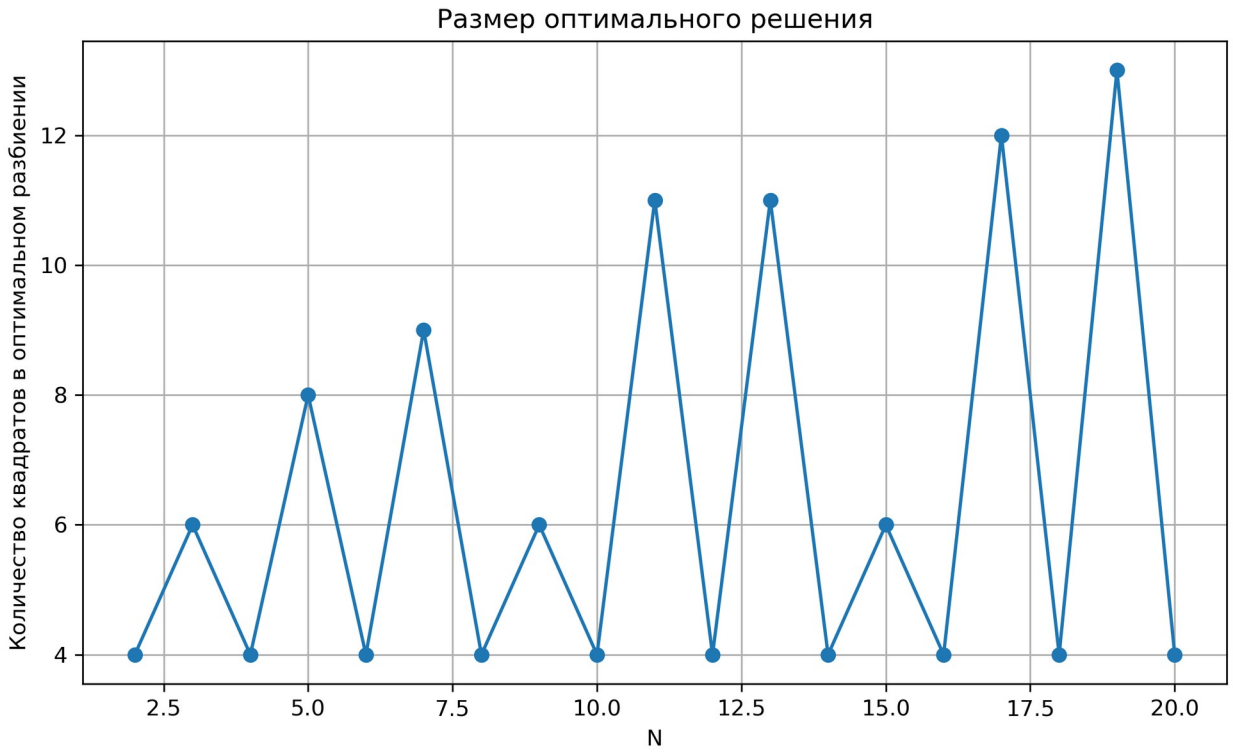

Рисунок 4. Тестирование.

Вывод.

В ходе лабораторной работы был реализован алгоритм разбиения квадрата на минимальное число квадратов с использованием метода поиска с возвратом. Алгоритм реализован итеративно, без использования рекурсии, что соответствует условиям задания. Для представления состояния использовался массив высот столбцов, что позволило сократить объём проверок по сравнению с двумерным представлением поля. Для ускорения работы были применены отсечения по текущему лучшему решению, нижней оценке и таблице посещённых состояний.

Приложения

Приложение А

lb1.py:

```
def first_info(h, n):
    min_h = h[0]
    pos = 0
    for i in range(1, n):
        v = h[i]
        if v < min_h:
            min_h = v
            pos = i

    run = 1
    i = pos + 1
    while i < n and h[i] == min_h:
        run += 1
        i += 1

    max_size = run
    down = n - min_h
    if max_size > down:
        max_size = down

    return pos, min_h, max_size


def greedy_bound(n):
    h = [0] * n
    placed = []

    while True:
        pos, min_h, max_size = first_info(h, n)
        if min_h == n:
            return placed

        s = max_size
        if not placed and s == n:
            s -= 1

        for j in range(pos, pos + s):
            h[j] += s

        placed.append((min_h, pos, s))


def solve(n):
    if n % 2 == 0:
        s = n // 2
        return [
            (0, 0, s),
            (0, s, s),
            (s, 0, s),
            (s, s, s)
        ]
```

```
    heights = [0] * n
    current = []
    best = greedy_bound(n)
    best_count = len(best)
    total = n * n
    filled = 0
    memo = {}

    pos, h, max_size = first_info(heights, n)
    if max_size == n:
        max_size -= 1

    stack = [[pos, h, max_size, n // 2 + 1, -1, -1, -1]]

    while stack:
        pos, h, next_size, min_size, pr, pc, ps = stack[-1]

        if next_size < min_size:
            stack.pop()
            if ps != -1:
                for j in range(pc, pc + ps):
                    heights[j] -= ps
                filled -= ps * ps
                current.pop()
            continue

        s = next_size
        stack[-1][2] = next_size - 1

        for j in range(pos, pos + s):
            heights[j] += s
        filled += s * s
        current.append((h, pos, s))
        used = len(current)

        prune = False

        if filled == total:
            if used < best_count:
                best_count = used
                best = current[:]
            prune = True
        elif used >= best_count:
            prune = True
        else:
            _, min_h, _ = first_info(heights, n)
            remain = total - filled
            max_possible = n - min_h
                low = (remain + max_possible * max_possible - 1) //
(max_possible * max_possible)

            if used + low >= best_count:
                prune = True
            else:
                key = tuple(heights)
                prev = memo.get(key)
                if prev is not None and prev <= used:
                    prune = True
```

```
                else:
                    memo[key] = used

        if prune:
            current.pop()
            filled -= s * s
            for j in range(pos, pos + s):
                heights[j] -= s
            continue

        pos2, h2, max_size2 = first_info(heights, n)
        stack.append([pos2, h2, max_size2, 1, h, pos, s])

    return best


n = int(input())
ans = solve(n)

print(len(ans))
for r, c, s in ans:
    print(r + 1, c + 1, s)
```

graphics.py

```
import time
import csv
import matplotlib.pyplot as plt


def first_info(h, n):
    min_h = h[0]
    pos = 0
    for i in range(1, n):
        if h[i] < min_h:
            min_h = h[i]
            pos = i

    run = 1
    i = pos + 1
    while i < n and h[i] == min_h:
        run += 1
        i += 1

    max_size = run
    down = n - min_h
    if max_size > down:
        max_size = down

    return pos, min_h, max_size


def greedy_bound(n):
    h = [0] * n
    placed = []

    while True:
```

```
        pos, min_h, max_size = first_info(h, n)
        if min_h == n:
            return placed

        s = max_size
        if not placed and s == n:
            s -= 1

        for j in range(pos, pos + s):
            h[j] += s

        placed.append((min_h, pos, s))


def solve_with_stats(n):
    stats = {
        "steps": 0,
        "prunes_best": 0,
        "prunes_bound": 0,
        "prunes_memo": 0,
        "states_pushed": 0,
        "states_popped": 0,
        "max_stack": 0,
        "memo_size": 0
    }

    if n % 2 == 0:
        s = n // 2
        ans = [
            (0, 0, s),
            (0, s, s),
            (s, 0, s),
            (s, s, s)
        ]
        return ans, stats

    heights = [0] * n
    current = []
    best = greedy_bound(n)
    best_count = len(best)
    total = n * n
    filled = 0
    memo = {}

    pos, h, max_size = first_info(heights, n)
    if max_size == n:
        max_size -= 1

    stack = [[pos, h, max_size, n // 2 + 1, -1, -1, -1]]
    stats["states_pushed"] += 1
    stats["max_stack"] = 1

    while stack:
        if len(stack) > stats["max_stack"]:
            stats["max_stack"] = len(stack)

        pos, h, next_size, min_size, pr, pc, ps = stack[-1]
```

```
        if next_size < min_size:
            stack.pop()
            stats["states_popped"] += 1
            if ps != -1:
                for j in range(pc, pc + ps):
                    heights[j] -= ps
                filled -= ps * ps
                current.pop()
            continue

        s = next_size
        stack[-1][2] = next_size - 1

        for j in range(pos, pos + s):
            heights[j] += s
        filled += s * s
        current.append((h, pos, s))
        used = len(current)
        stats["steps"] += 1

        prune = False

        if filled == total:
            if used < best_count:
                best_count = used
                best = current[:]
            prune = True
        elif used >= best_count:
            stats["prunes_best"] += 1
            prune = True
        else:
            _, min_h, _ = first_info(heights, n)
            remain = total - filled
            max_possible = n - min_h
                low = (remain + max_possible * max_possible - 1) //
(max_possible * max_possible)

            if used + low >= best_count:
                stats["prunes_bound"] += 1
                prune = True
            else:
                key = tuple(heights)
                prev = memo.get(key)
                if prev is not None and prev <= used:
                    stats["prunes_memo"] += 1
                    prune = True
                else:
                    memo[key] = used

        if prune:
            current.pop()
            filled -= s * s
            for j in range(pos, pos + s):
                heights[j] -= s
            continue

        pos2, h2, max_size2 = first_info(heights, n)
        stack.append([pos2, h2, max_size2, 1, h, pos, s])
```

```
        stats["states_pushed"] += 1

    stats["memo_size"] = len(memo)
    return best, stats


def run_experiment(n_start=2, n_end=20, repeats=5):
    data = {
        "n": [],
        "time_avg": [],
        "time_min": [],
        "time_max": [],
        "steps": [],
        "prune_best": [],
        "prune_bound": [],
        "prune_memo": [],
        "states_pushed": [],
        "states_popped": [],
        "max_stack": [],
        "memo_size": [],
        "answer_sizes": []
    }

    print(
        "N | time_avg | time_min | time_max | steps | prune_best | "
         "prune_bound | prune_memo | pushed | popped | max_stack | memo
| answer"
    )

    for n in range(n_start, n_end + 1):
        times = []
        ans = None
        last_stats = None

        for _ in range(repeats):
            start = time.perf_counter()
            ans, stats = solve_with_stats(n)
            finish = time.perf_counter()
            times.append(finish - start)
            last_stats = stats

        avg_time = sum(times) / repeats
        min_time = min(times)
        max_time = max(times)

        data["n"].append(n)
        data["time_avg"].append(avg_time)
        data["time_min"].append(min_time)
        data["time_max"].append(max_time)
        data["steps"].append(last_stats["steps"])
        data["prune_best"].append(last_stats["prunes_best"])
        data["prune_bound"].append(last_stats["prunes_bound"])
        data["prune_memo"].append(last_stats["prunes_memo"])
        data["states_pushed"].append(last_stats["states_pushed"])
        data["states_popped"].append(last_stats["states_popped"])
        data["max_stack"].append(last_stats["max_stack"])
        data["memo_size"].append(last_stats["memo_size"])
        data["answer_sizes"].append(len(ans))
```

```
        print(
            f"{n:2d} | "
            f"{avg_time:8.6f} | "
            f"{min_time:8.6f} | "
            f"{max_time:8.6f} | "
            f"{last_stats['steps']:5d} | "
            f"{last_stats['prunes_best']:10d} | "
            f"{last_stats['prunes_bound']:11d} | "
            f"{last_stats['prunes_memo']:10d} | "
            f"{last_stats['states_pushed']:6d} | "
            f"{last_stats['states_popped']:6d} | "
            f"{last_stats['max_stack']:9d} | "
            f"{last_stats['memo_size']:4d} | "
            f"{len(ans):6d}"
        )

    return data


def save_csv(data, filename="results.csv"):
    keys = list(data.keys())
    rows = zip(*(data[key] for key in keys))

    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f, delimiter=";")
        writer.writerow(keys)
        writer.writerows(rows)


def plot_time(data):
    plt.figure(figsize=(8, 5))
      plt.plot(data["n"], data["time_avg"], marker="o", label="Среднее
время")
                plt.plot(data["n"],    data["time_min"],    marker="o",
label="Минимальное время")
                plt.plot(data["n"],    data["time_max"],    marker="o",
label="Максимальное время")
    plt.xlabel("N")
    plt.ylabel("Время, сек")
    plt.title("Зависимость времени работы от N")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig("graph_time.png", dpi=300, bbox_inches="tight")
    plt.show()


def plot_steps(data):
    plt.figure(figsize=(8, 5))
    plt.plot(data["n"], data["steps"], marker="o")
    plt.xlabel("N")
    plt.ylabel("Количество шагов")
    plt.title("Количество шагов перебора")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("graph_steps.png", dpi=300, bbox_inches="tight")
    plt.show()
```

```
def plot_stack(data):
    plt.figure(figsize=(8, 5))
    plt.plot(data["n"], data["max_stack"], marker="o")
    plt.xlabel("N")
    plt.ylabel("Максимальная глубина стека")
    plt.title("Максимальная глубина стека состояний")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("graph_stack.png", dpi=300, bbox_inches="tight")
    plt.show()


def plot_answer_sizes(data):
    plt.figure(figsize=(8, 5))
    plt.plot(data["n"], data["answer_sizes"], marker="o")
    plt.xlabel("N")
    plt.ylabel("Количество квадратов в оптимальном разбиении")
    plt.title("Размер оптимального решения")
    plt.grid(True)
    plt.tight_layout()
                      plt.savefig("graph_answer_sizes.png",     dpi=300,
bbox_inches="tight")
    plt.show()


def plot_states(data):
    plt.figure(figsize=(8, 5))
            plt.plot(data["n"],   data["states_pushed"],   marker="o",
label="Добавлено в стек")
            plt.plot(data["n"],   data["states_popped"],   marker="o",
label="Удалено из стека")
    plt.xlabel("N")
    plt.ylabel("Количество состояний")
    plt.title("Работа со стеком состояний")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig("graph_states.png", dpi=300, bbox_inches="tight")
    plt.show()


def plot_memo(data):
    plt.figure(figsize=(8, 5))
    plt.plot(data["n"], data["memo_size"], marker="o")
    plt.xlabel("N")
    plt.ylabel("Количество записей в memo")
    plt.title("Размер таблицы посещённых состояний")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("graph_memo.png", dpi=300, bbox_inches="tight")
    plt.show()



data = run_experiment(2, 20, repeats=5)
```

```
save_csv(data, "results.csv")

plot_time(data)
plot_steps(data)
plot_stack(data)
plot_answer_sizes(data)
plot_states(data)
plot_memo(data)
```